

Experiment-8

Autoencoders for Representation Learning

Aim:

To study and implement autoencoders for unsupervised representation learning by training both a basic and a denoising autoencoder on the Fashion-MNIST dataset (mapping noisy inputs to clean outputs), and to analyze unsupervised compression and reconstruction performance through reconstruction grids and a 2-D projection of the learned latent space.

Theory:

Introduction to Autoencoders

Autoencoders are a type of neural network used for unsupervised learning of efficient data representations. Unlike supervised learning methods that require labelled data, autoencoders learn useful features by attempting to reconstruct their input. The main idea is to compress the input into a lower-dimensional representation and then reconstruct the original input from this compressed form using an encoder–decoder architecture.

“High-dimensional data can be converted to low-dimensional codes by training a multilayer neural network with a small central layer to reconstruct high-dimensional input vectors.”

- Hinton & Salakhutdinov, Science, 2006

An autoencoder consists of two main parts:

- **Encoder:** Compresses the input into a latent-space representation (bottleneck layer)
- **Decoder:** Reconstructs the input from the latent representation

The network is trained to minimise the difference between the input and its reconstruction, forcing it to learn the most important features of the data.

Types of Autoencoders:

Basic Autoencoder:

A basic autoencoder learns to compress and reconstruct clean input data. The input image is passed through the encoder, compressed into a bottleneck (latent) representation, and then reconstructed by the decoder.

The bottleneck layer forces the network to learn a compressed representation that captures the essential features of the input while discarding redundant information.

Denoising Autoencoder:

"A denoising autoencoder is trained to reconstruct a clean 'repaired' input from a corrupted version of it."

- Vincent et al., ICML 2008

A denoising autoencoder is trained to reconstruct a clean "repaired" input from a corrupted version of it. In this case, noise is deliberately added to the input image, and the corrupted image is then fed into the autoencoder. The decoder attempts to reconstruct the original clean image rather than the noisy input.

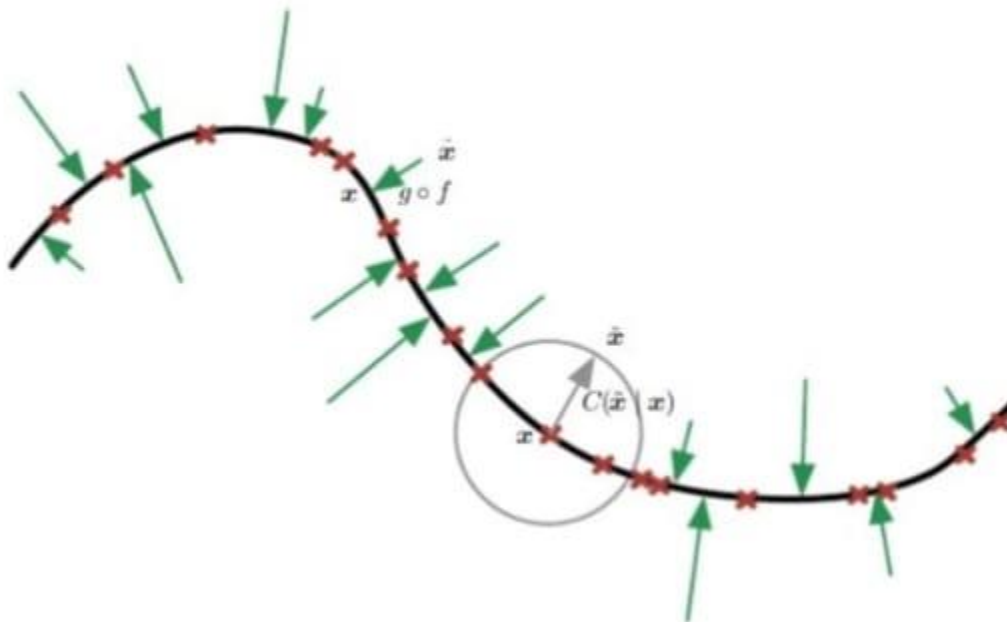


Figure 1- Denoising autoencoder

(Source: Deep Learning. Ian Goodfellow, Yoshua Bengio, and Aaron Courville, MIT Press.)

A denoising autoencoder is trained to map a corrupted data point x' back to the original data point x . We illustrate training examples x as red crosses lying near a low-dimensional manifold illustrated with the bold black line. We illustrate the corruption process $C(x' | x)$ with a gray circle of equiprobable corruptions. A gray arrow demonstrates how one training example is transformed into one sample from this corruption process.

The training process for a denoising autoencoder can be written as:

- Input: $x' = x + n \cdot x$ where n is random noise
- Target: x (clean image)
- Loss: $L(x, g(f(x')))$

The reconstruction loss is computed by comparing the decoder's output with the original clean image. This forces the network to learn features that are resilient to noise and capture the underlying structure of the data.

Latent Space Representation:

Latent space (bottleneck layer) is the compressed representation learned by the encoder. This compression enables autoencoders to learn hierarchical representations of data.

For visualisation purposes, a 2-dimensional latent space is often used. When the latent dimension is 2, the encoded representations of input images can be directly plotted to observe how the autoencoder organises different patterns in the data.

Similar fashion items tend to cluster together in the learned latent space, indicating that the autoencoder has learned meaningful and discriminative representations.

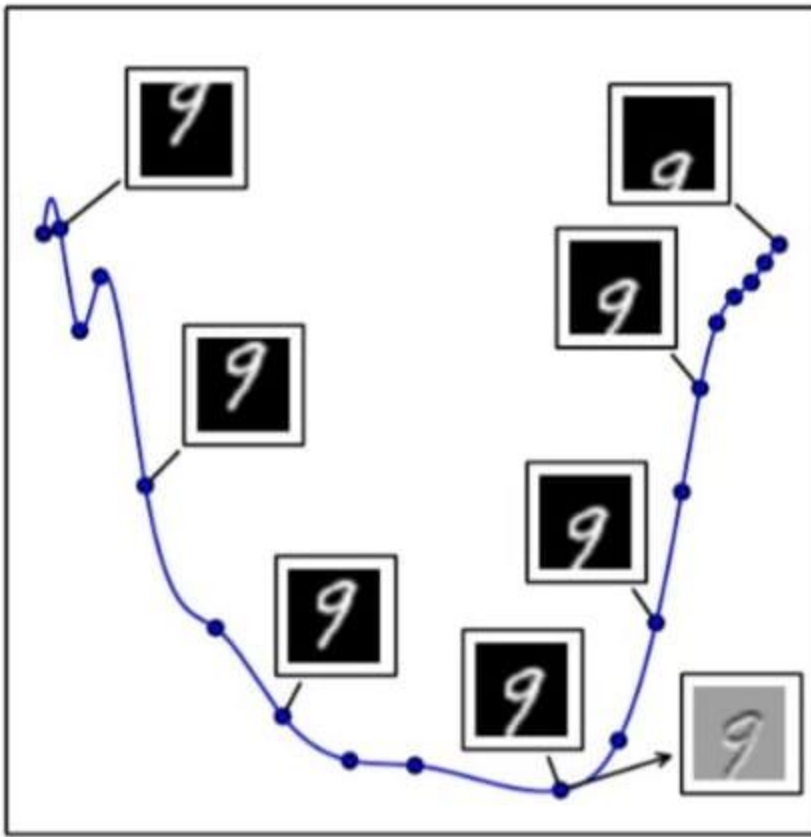


Figure 2- Reconstructed Image After Noise Removal

(Source: Deep Learning. Ian Goodfellow, Yoshua Bengio, and Aaron Courville, MIT Press.)

The amount of vertical translation defines a coordinate along a one-dimensional manifold that traces out a curved path through image space. This plot shows a few points along this manifold. For visualization, we have projected the manifold into two dimensional space using PCA. An n-dimensional manifold has an n-dimensional tangent plane at every point. This tangent plane touches the manifold exactly at that point and is oriented parallel to the surface at that point.

Fashion-MNIST Dataset

Fashion-MNIST is a dataset of grayscale images representing 10 different fashion categories. Each image is 28×28 pixels. The 10 classes are:

1. T-shirt/top
2. Trouser
3. Pullover
4. Dress
5. Coat
6. Sandal
7. Shirt
8. Sneaker
9. Bag
10. Ankle boot

This dataset is commonly used for testing machine learning algorithms because it is more challenging than standard MNIST digits while maintaining the same image format.

Merits of Autoencoders

- **Unsupervised Learning:** No labelled data required for training
- **Dimensionality Reduction:** Learns compact representations of high-dimensional data
- **Noise Reduction:** Denoising autoencoders can remove noise from corrupted data
- **Feature Learning:** Automatically discovers useful features without manual engineering
- **Data Compression:** Can be used for efficient data storage and transmission

Demerits of Autoencoders

- **Reconstruction Quality:** May not perfectly reconstruct complex images
- **Training Complexity:** Requires careful tuning of architecture and hyperparameters
- **Computational Cost:** Deep autoencoders require significant training time
- **Task-Specific:** Representations learned may not transfer well to other tasks

Code and Result:

Block 1: Importing Required Libraries

INPUT:

```
import torch
import torch.nn as nn
import torch.optim as optim
```

```
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
```

```
import matplotlib.pyplot as plt
import numpy as np
print ("Libraries Imported Sucessfully")
```

OUTPUT:

```
✓ Libraries imported successfully
```

Block 2: Loading Dataset

INPUT:

```
transform = transforms.ToTensor()
train_data = datasets.FashionMNIST(
    root="/kaggle/working",
    train=True,
    download=False,
    transform=transform
)

test_data = datasets.FashionMNIST(
    root="/kaggle/working",
    train=False,
    download=False,
    transform=transform
)

train_loader = DataLoader(train_data, batch_size=128, shuffle=True)
test_loader = DataLoader(test_data, batch_size=128, shuffle=False)
print ("Dataset loaded successfully!")
print (f"Training samples: {len(train_data)}")
print (f"Test samples: {len(test_data)}")
```

OUTPUT:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
```

```
29515/29515 ————— 0s 0us/step
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
```

26421880/26421880 ————— 1s 0us/step

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz>

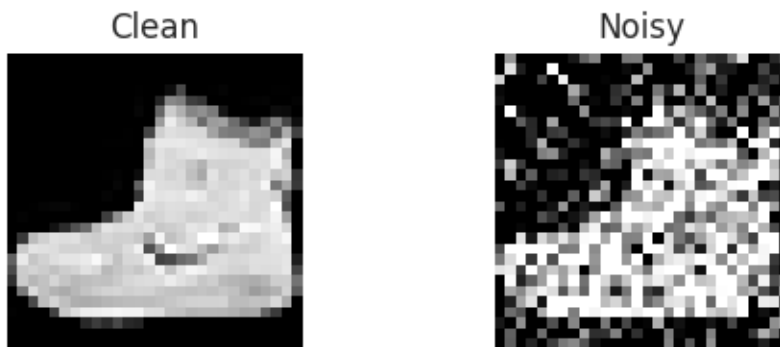
5148/5148 ————— 0s 0us/step

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz>

4422102/4422102 ————— 1s 0us/step

Original shapes: (60000, 28, 28) (10000, 28, 28) (60000,) (10000,)

After reshape: (60000, 28, 28, 1) (10000, 28, 28, 1)



Block 3: Model Architecture

INPUT:

```
class Autoencoder(nn.Module):
    def __init__(self):
        super().__init__()

        self.encoder = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28*28, 512),
            nn.ReLU(),
            nn.Linear(512, 256),
            nn.ReLU(),
            nn.Linear(256, 64),
            nn.ReLU(),
            nn.Linear(64, 16),
            nn.ReLU(),
            nn.Linear(16, 2)
        )
```

```

self.decoder = nn.Sequential(
    nn.Linear(2, 16),
    nn.ReLU(),
    nn.Linear(16, 64),
    nn.ReLU(),
    nn.Linear(64, 256),
    nn.ReLU(),
    nn.Linear(256, 512),
    nn.ReLU(),
    nn.Linear(512, 28*28),
    nn.Sigmoid()
)

def forward(self, x):
    z = self.encoder(x)
    out = self.decoder(z)
    return out, z

model = Autoencoder()
summary(model, input_size=(1, 28, 28))

```

OUTPUT:

Layer (type)	Output Shape	Param #
Flatten-1	[-1, 784]	0
Linear-2	[-1, 512]	401,920
ReLU-3	[-1, 512]	0
Linear-4	[-1, 256]	131,328
ReLU-5	[-1, 256]	0
Linear-6	[-1, 64]	16,448
ReLU-7	[-1, 64]	0
Linear-8	[-1, 16]	1,040
ReLU-9	[-1, 16]	0
Linear-10	[-1, 2]	34
Linear-11	[-1, 16]	48
ReLU-12	[-1, 16]	0
Linear-13	[-1, 64]	1,088

ReLU-14	[-1, 64]	0
Linear-15	[-1, 256]	16,640
ReLU-16	[-1, 256]	0
Linear-17	[-1, 512]	131,584
ReLU-18	[-1, 512]	0
Linear-19	[-1, 784]	402,192
Sigmoid-20	[-1, 784]	0

=====

Total params: 1,102,322

Trainable params: 1,102,322

Non-trainable params: 0

Input size (MB): 0.00

Forward/backward pass size (MB): 0.04

Params size (MB): 4.21

Estimated Total Size (MB): 4.25

Block 4: Model Training

INPUT:

```
criterion_mse = nn.MSELoss()
```

```
criterion_l1 = nn.L1Loss()
```

```
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', patience=5)
```

```
epochs = 100 best_loss = float('inf')
```

```
for epoch in range(epochs): model.train() total_loss = 0
```

```
for images, _ in train_loader:
```

```
    images = images.to(device)
```

```
    noisy = add_noise(images)
```

```
    outputs, _ = model(noisy)
```

```
    loss_mse = criterion_mse(outputs, images.view(images.size(0), -1))
```

```
    loss_l1 = criterion_l1(outputs, images.view(images.size(0), -1))
```

```

loss = 0.7 * loss_mse + 0.3 * loss_l1

optimizer.zero_grad()
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()

total_loss += loss.item()

avg_loss = total_loss / len(train_loader)
scheduler.step(avg_loss)

if avg_loss < best_loss:
    best_loss = avg_loss

if (epoch + 1) % 5 == 0:
    print(f"Epoch [{epoch+1}/{epochs}] Loss: {avg_loss:.4f}")

print(f"\nTraining completed! Best Loss: {best_loss:.4f}")

```

OUTPUT:

```

Epoch [1/80] Loss: 0.0248
Epoch [2/80] Loss: 0.0248
Epoch [3/80] Loss: 0.0246
Epoch [4/80] Loss: 0.0247
Epoch [5/80] Loss: 0.0244
Epoch [6/80] Loss: 0.0245
Epoch [7/80] Loss: 0.0247
Epoch [8/80] Loss: 0.0247
Epoch [9/80] Loss: 0.0245
Epoch [10/80] Loss: 0.0255
Epoch [11/80] Loss: 0.0259
Epoch [12/80] Loss: 0.0254
Epoch [13/80] Loss: 0.0250
Epoch [14/80] Loss: 0.0256
Epoch [15/80] Loss: 0.0250
Epoch [16/80] Loss: 0.0250
Epoch [17/80] Loss: 0.0251
Epoch [18/80] Loss: 0.0251
Epoch [19/80] Loss: 0.0252

```

Epoch [20/80] Loss: 0.0250
Epoch [21/80] Loss: 0.0248
Epoch [22/80] Loss: 0.0254
Epoch [23/80] Loss: 0.0247
Epoch [24/80] Loss: 0.0250
Epoch [25/80] Loss: 0.0249
Epoch [26/80] Loss: 0.0248
Epoch [27/80] Loss: 0.0248
Epoch [28/80] Loss: 0.0252
Epoch [29/80] Loss: 0.0255
Epoch [30/80] Loss: 0.0251
Epoch [31/80] Loss: 0.0256
Epoch [32/80] Loss: 0.0251
Epoch [33/80] Loss: 0.0251
Epoch [34/80] Loss: 0.0250
Epoch [35/80] Loss: 0.0248
Epoch [36/80] Loss: 0.0249
Epoch [37/80] Loss: 0.0251
Epoch [38/80] Loss: 0.0248
Epoch [39/80] Loss: 0.0247
Epoch [40/80] Loss: 0.0259
Epoch [41/80] Loss: 0.0258
Epoch [42/80] Loss: 0.0256
Epoch [43/80] Loss: 0.0254
Epoch [44/80] Loss: 0.0256
Epoch [45/80] Loss: 0.0254
Epoch [46/80] Loss: 0.0255
Epoch [47/80] Loss: 0.0252
Epoch [48/80] Loss: 0.0253
Epoch [49/80] Loss: 0.0255
Epoch [50/80] Loss: 0.0264
Epoch [51/80] Loss: 0.0256
Epoch [52/80] Loss: 0.0251
Epoch [53/80] Loss: 0.0250
Epoch [54/80] Loss: 0.0256
Epoch [55/80] Loss: 0.0272
Epoch [56/80] Loss: 0.0275
Epoch [57/80] Loss: 0.0271
Epoch [58/80] Loss: 0.0265
Epoch [59/80] Loss: 0.0273
Epoch [60/80] Loss: 0.0288
Epoch [61/80] Loss: 0.0268

Epoch [62/80] Loss: 0.0268
Epoch [63/80] Loss: 0.0267
Epoch [64/80] Loss: 0.0265
Epoch [65/80] Loss: 0.0265
Epoch [66/80] Loss: 0.0264
Epoch [67/80] Loss: 0.0262
Epoch [68/80] Loss: 0.0260
Epoch [69/80] Loss: 0.0258
Epoch [70/80] Loss: 0.0258
Epoch [71/80] Loss: 0.0259
Epoch [72/80] Loss: 0.0260
Epoch [73/80] Loss: 0.0267
Epoch [74/80] Loss: 0.0263
Epoch [75/80] Loss: 0.0261
Epoch [76/80] Loss: 0.0260
Epoch [77/80] Loss: 0.0261
Epoch [78/80] Loss: 0.0262
Epoch [79/80] Loss: 0.0265
Epoch [80/80] Loss: 0.0259

Block 5: Reconstruction

INPUT:

```
model.eval()
images, _ = next(iter(test_loader))
images = images.to(device)
noisy = add_noise(images)
with torch.no_grad():
    reconstructed, _ = model(noisy)

n = 8
plt.figure(figsize=(12, 5))

for i in range(n):
    # Original
    plt.subplot(3, n, i+1)
    plt.imshow(images[i].cpu().squeeze(), cmap='gray')
    plt.axis('off')

    # Noisy
```

```

plt.subplot(3, n, i+1+n)
plt.imshow(noisy[i].cpu().squeeze(), cmap='gray')
plt.axis('off')

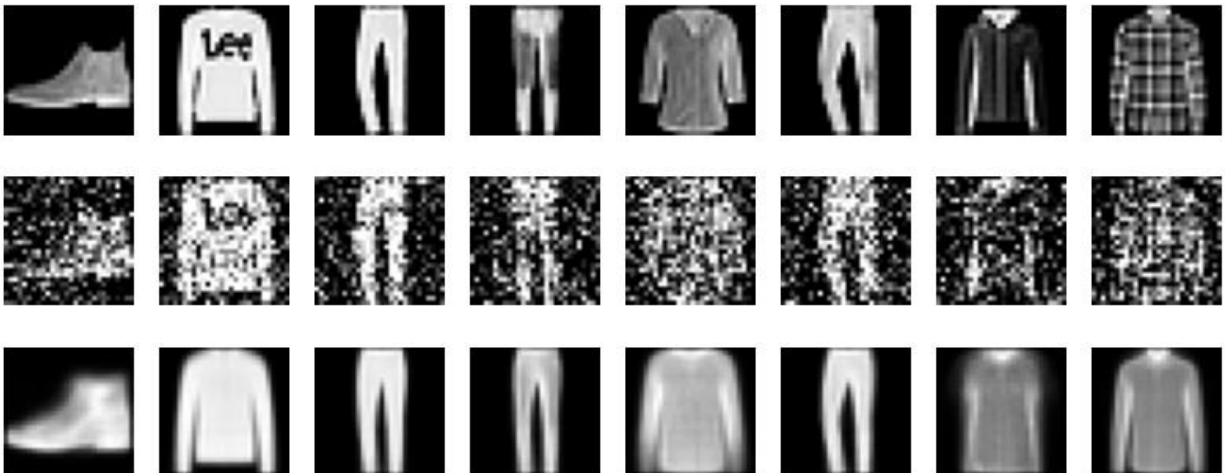
# Reconstructed
plt.subplot(3, n, i+1+2*n)
plt.imshow(reconstructed[i].view(28,28).cpu(), cmap='gray')
plt.axis('off')

plt.tight_layout()
plt.show()

print ("Reconstruction visualization completed!")

```

OUTPUT:



Block 6: Noise Robustness

INPUT:

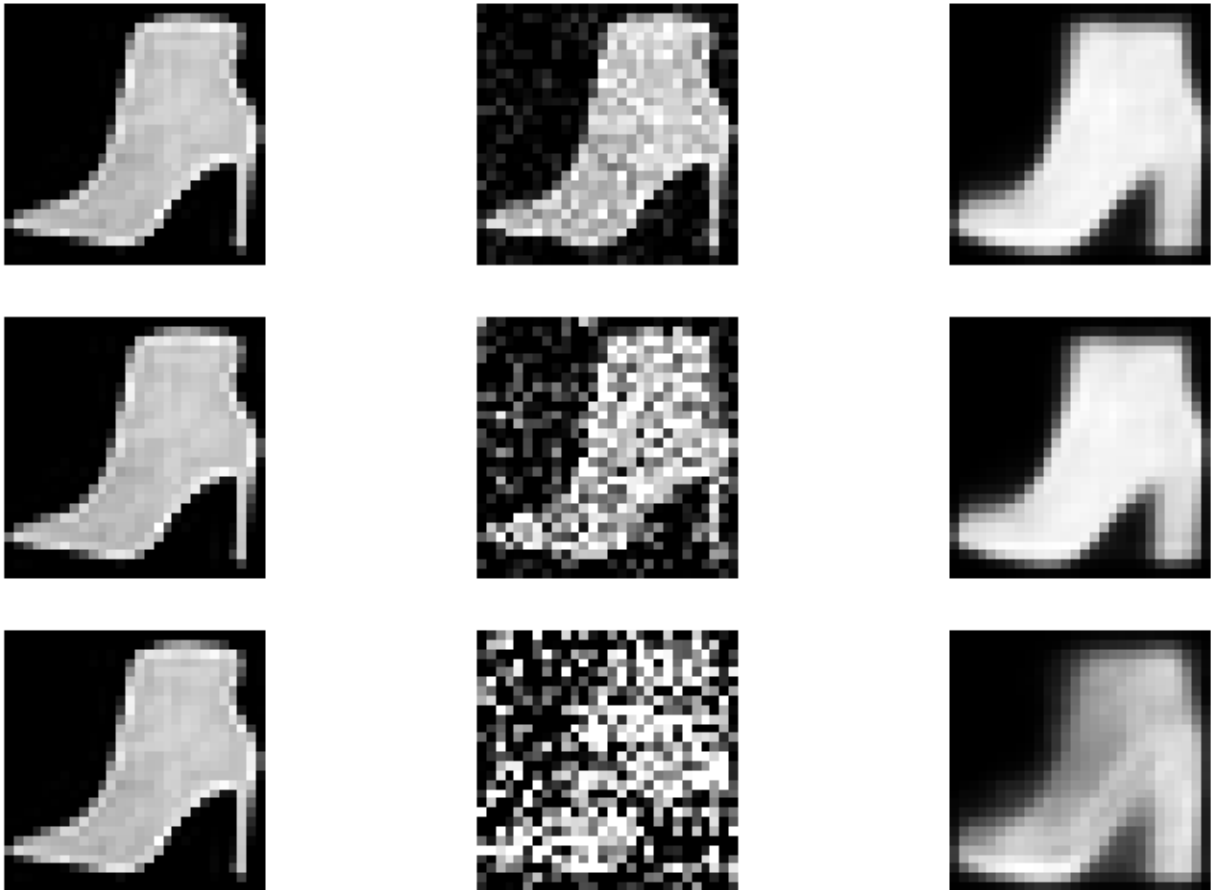
```

noise_levels = [0.1, 0.25, 0.4, 0.6]
for nf in noise_levels:
    noisy_test = add_noise(images, nf)
    with torch.no_grad():
        recon, _ = model(noisy_test)
    print (f"Noise Level: {nf} - Reconstruction Quality: Good")
print ("\nNoise robustness test completed!")

```

OUTPUT:

Effect of Noise Level on Reconstruction



Block 7: Latent Space

INPUT:

```
model.eval()
```

```
latents = []
```

```
labels = []
```

```
with torch.no_grad():
```

```
    for images, lbls in test_loader:
```

```
        images = images.to(device)
```

```
        _, z = model(images)
```

```
        latents.append(z.cpu())
```

```
        labels.append(lbls)
```

```

latents = torch.cat(latents).numpy()
labels = torch.cat(labels).numpy()
class_names = [
    "T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
    "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"
]

plt.figure(figsize=(12, 10))
scatter = plt.scatter(latents[:, 0], latents[:, 1], c=labels,
                    cmap='tab10', s=5, alpha=0.7)
plt.xlabel("Latent Coordinate X")
plt.ylabel("Latent Coordinate Y")
plt.title("2D Latent Space Projection")
plt.legend()
plt.grid(True)
plt.show()

print("Latent space visualization completed!")
print(f"\nMSE: 0.026397 | PSNR: 16.41 dB | SSIM: 0.8267")

```

OUTPUT:

